A3-PS017-TC461

Tail-Latency Prediction for URLLC-Like Traffic in Simulation

Alan Shinoj
Personal : alanshinoj432@gmail.com
Albhy Mathew
Personal : albhymatheww@gmail.com
Dr. M. Manicka Raja
Personal : manickaraja@karunya.edu

AIORI Hackathon 3

October 5, 2026

Problem Description

- **The problem**: URLLC-like traffic requires packets to meet strict delivery deadlines. When latency-sensitive packets share a queue with competing bulk traffic, queueing delays can increase and cause deadline violations. The problem gets worse when the link itself is unreliable and its service rate drops for short, unpredictable periods.
- **Why averages are insufficient**: Mean latency can hide rare but important delays. The 95th-, 99th- and 99.9th-percentile delays, together with the deadline-miss rate, give a clearer view of tail behavior.
- **The challenge**: A FIFO scheduler ignores deadlines. Strict priority protects URLLC packets, but it reorders the queue for every URLLC arrival, including the many packets that would have met their deadline anyway, so bulk traffic pays a cost even when nothing was at risk.
- **Our approach**: A calibrated, minimal-intervention scheduler. It acts only when it predicts, with a stated confidence level, that a URLLC packet will miss its deadline and that acting can still save it. It then moves the packet only as far forward as needed to meet the deadline.
- **Our research question**: Can a calibrated delay predictor, trained to identify URLLC packets that would miss their deadline and can still be saved, reach a deadline-miss rate close to strict priority while intervening far less often and imposing a smaller cost on bulk traffic than strict priority, Earliest

Deadline First (EDF), and an analytical deadline-risk rule?

- **Scope**: A reproducible single-queue simulation with a time-varying link, two main traffic-load conditions and one unseen stress condition. The study evaluates simulated queueing behavior and does not claim real-world 5G/6G reliability or radio-interface guarantees.

Background

- **Queueing and tail latency**: When packets arrive faster than the available service capacity for sustained periods, the queue grows and waiting times increase. URLLC targets are defined on the tail, for example a reliability requirement on delivering small packets within a strict latency bound [1].

- **Competing traffic**: Large or numerous bulk packets can delay smaller deadline-sensitive packets when both share a FIFO queue.

- **Deadline risk**: A packet's completion time depends on the remaining work of the packet in service, the workload ahead of it, its own service requirement, the link's service rate, and its deadline.

- **Why a learned predictor is needed**: In a FIFO queue with a constant, known service rate, packets that arrive later never delay earlier ones, so a packet's completion time is fixed the moment it arrives and a simple formula computes it exactly. We therefore model a realistic source of uncertainty: a time-varying link following the Gilbert–Elliott

model [2], [3], with a hidden "good" state and a "bad" state that has a much lower service rate. The scheduler cannot see this state; it sees only the service times of recent packets. An analytical rule that assumes the average rate underestimates delay during bad periods, while a learned model can infer the likely state from recent service history.

- **Study boundary**: The simulator represents a simplified queueing system. It does not model the complete 5G/6G radio stack or establish standards compliance.

Proposed Optimization

- **Calibrated risk prediction:** At each URLLC arrival, a quantile regression model predicts an upper bound on the packet's delay. Conformal calibration [4], [5] adjusts this bound so that the true delay stays below it with a chosen probability (for example 99%), so the bound's reliability is measured and stated rather than assumed.
- **Savability-aware triggering:** Some packets at risk cannot be saved even with priority, and acting on them only harms bulk traffic. Because we control the simulator, we run each situation twice, with and without the intervention, and label whether the intervention changes a miss into a deadline met. A classifier trained on these counterfactual labels, an approach related to uplift modelling [6], predicts whether acting will help.
- **Minimal intervention:** When the scheduler does act, it inserts the URLLC packet at the latest position in the queue that still meets its deadline (just-in-time insertion), rather than at the head. Bulk packets ahead of that point are not disturbed.
- **Measured trade-off:** Every policy is evaluated on both URLLC performance and the cost to bulk traffic. Results are presented as a trade-off curve and summarized by a single metric, the cost per saved packet.

Literature & RFC Review

- **RFC 2330, Framework for IP Performance Metrics [7]**: Provides a general framework for defining and reporting IP performance measurements. We follow its terminology for metrics but do not claim standards compliance for the simulation.
- **RFC 7679, A One-Way Delay Metric for IPPM [8]**: Defines one-way packet delay. In our simulator, delay is defined as departure time minus arrival time at the simulated queue (queueing plus transmission time), reported as percentiles.
- **3GPP TR 38.913 [1]**: Provides the standards context for URLLC latency and reliability targets. We use these concepts to set packet sizes and deadlines, not to claim radio-level compliance.
- **Bufferbloat, Gettys and Nichols [9], and CoDel, RFC 8289 [10]**: Explain how excessive buffering increases delay, and control queue delay by dropping packets when sojourn time stays high. CoDel works on aggregate queue delay and does not consider individual packet deadlines.

- **L4S, RFC 9330 [11]:** Achieves low queueing delay with a separate low-latency queue and scalable congestion control. It relies on cooperation from endpoints, whereas our scheduler acts locally at the queue.

- **DetNet, RFC 8655 [12]:** Provides bounded latency through resource reservation and explicit routes. This requires advance provisioning; our work targets shared queues without reservation.

- **Deadline and fair scheduling [13], [14]**: EDF is optimal for meeting deadlines on a single preemptive processor [13]. Packet transmission is non-preemptive, so we use EDF as a strong baseline rather than an optimum. Fair queueing [14] isolates flows but does not consider deadlines.

- **Learning-based scheduling [15]:** Deep reinforcement learning has been applied widely to networking. It can learn complex policies but is harder to interpret and gives no statistical guarantee. We choose supervised prediction with conformal calibration because it gives a stated coverage level and one interpretable tuning parameter.

- **Conformal prediction [4], [5], [16]:** Produces prediction bounds with coverage guarantees that do not depend on the data distribution. Adaptive conformal inference [16] keeps the coverage level when the data distribution drifts, which matters when traffic load changes.

- **Research gap investigated:** Existing approaches either act on every packet (strict priority, EDF), act on aggregate delay (CoDel), or require provisioning or endpoint support (DetNet, L4S). This project tests whether a calibrated, per-packet risk predictor that triggers minimal interventions only when they are likely to help can match the reliability of always-on prioritization at a lower cost to competing traffic.

- **Evaluation principle:** Compare all policies on the same traffic scenarios and random seeds, and report both latency-sensitive performance and the cost imposed on competing traffic.

.

Proposed Solution

Objective: Build a reproducible discrete-event simulator and use it to evaluate whether calibrated, savability-aware deadline-risk prediction with minimal intervention improves the trade-off between URLLC reliability and bulk-traffic cost.

1. Simulation Environment

- Model one finite-buffer, single-server, non-preemptive queue.
- Model the link as a two-state Gilbert–Elliott channel. Each state has its own service rate, switching between states is random, and the state is hidden from the scheduler.
- Generate URLLC-like traffic with Poisson arrivals, small fixed-size packets and an explicit deadline per packet.
- Generate bulk traffic with bursty on/off arrivals, large packets and no deadline.
- Evaluate two main load conditions, moderate (average utilization about 0.7) and heavy (about 0.95), plus one unseen stress condition with different burstiness, used only for testing generalization.
- Fix all parameters in a configuration file before evaluation begins.

- Validate the simulator by holding the channel in one state and comparing its mean delay against the analytical M/G/1 result at the same load.

2. Prediction Model

- **Features available at arrival time:** bytes and packets ahead in the queue, remaining bytes of the packet in service, the packet's own size, deadline slack, and statistics of recent service rates (latest rate, moving average, variance, time since the last observed rate change). The hidden channel state is never used as a feature.
- **Delay bound model:** a quantile regression model (gradient-boosted trees, with linear quantile regression as a fallback) predicts a high quantile of the packet's delay under FIFO.
- **Calibration:** conformalized quantile regression [5] on a held-out calibration window turns the prediction into an upper bound with target coverage $1 - \alpha$. Adaptive conformal inference [16] updates the bound online when load changes.
- **Savability model:** a lightweight classifier trained on counterfactual labels. At sampled URLLC arrivals, the simulator state is copied and run forward twice, once with FIFO and once with the intervention. The label is "saveable" when the intervention turns a miss into a deadline met.

- **Data split:** chronological, with training (60%), calibration (15%) and test (25%) windows, plus independent random seeds for final testing.

3. Scheduling Policies

- **FIFO:** packets served in arrival order.
- **Strict priority:** every URLLC packet is served before any waiting bulk packet.
- **EDF:** packets served in order of deadline, with bulk packets given a large nominal deadline.
- **Analytical rule + just-in-time insertion:** completion time estimated with the average service rate; intervention triggered when the estimate exceeds the deadline.
- **Learned bound + just-in-time insertion:** the calibrated bound triggers the intervention.
- **Full system:** intervention triggered only when the calibrated bound exceeds the deadline and the savability model predicts the packet can be saved.

The analytical rule, the learned bound and the full system all use the same action, just-in-time insertion, so differences between them come only from the trigger. Insertion is non-preemptive: the packet in service always finishes.

4. Evaluation

- **URLLC performance:** p95, p99 and p99.9 delay, and deadline-miss rate, with 95% bootstrap confidence intervals across seeds.
- **Prediction quality:** empirical coverage of the bound against its target, bound width, and precision, recall, false-positive rate and PR-AUC for miss detection. PR-AUC is used because deadline misses are rare.
- **Intervention efficiency:** number of interventions and the share of interventions that changed the outcome.
- **Bulk traffic impact:** throughput, p99 delay and packet loss.
- **Cost per saved packet:** additional bulk delay (in byte-seconds) per URLLC deadline saved, relative to FIFO.
- **Trade-off curve:** URLLC miss rate against bulk cost for every policy, with α and the savability threshold swept for the learned policies.
- **Generalization:** train under one load condition and test under the unseen stress condition.

5. Hypotheses

- **H1:** Under the bursty channel, the learned bound detects deadline misses with higher recall than the analytical rule at the same false-positive rate.
- **H2:** The full system reaches a URLLC deadline-miss rate close to strict priority while intervening less often and with a lower cost per saved packet than strict priority and EDF.
- **H3:** The calibrated bound keeps its target coverage on held-out seeds, and adaptive calibration keeps it under the unseen stress condition.
- **H4:** Savability filtering reduces the share of interventions that do not change the outcome.

Scope: The project evaluates a simplified queueing model; it does not claim end-to-end 5G/6G or radio-interface guarantees.

Optimization Proposed

The optimization replaces "always prioritize" with "prioritize only when it is necessary and useful, and only as much as needed." It is applied on top of a FIFO baseline, which remains the default behavior. The enhancements are:

- **Enhancement 1, Hidden-State Awareness:** FIFO and the analytical rule ignore changes in link quality. Our feature extractor summarizes recent service times so the model can infer whether the link is currently in a slow period.
- **Enhancement 2, Calibrated Delay Bound:** Instead of a yes/no guess, the system predicts an upper bound on delay with a stated coverage level. The parameter α maps directly to a reliability target and is the system's single main tuning parameter.
- **Enhancement 3, Savability-Aware Trigger:** Counterfactual labels from the simulator let the system skip packets that are either safe or cannot be saved, so bulk traffic is not disturbed without benefit.
- **Enhancement 4, Just-in-Time Insertion:** An at-risk packet is placed at the latest queue position that still meets its deadline, rather than at the head of the queue.
- **Enhancement 5, Trade-off Measurement:** Results are reported as a trade-off curve against all baselines, together with the cost per saved packet, so the cost of the improvement is stated explicitly.

Before vs. After Comparison

Aspect	Before	After
Scheduling Policy	FIFO ignores deadlines; strict priority reorders for every URLLC packet	FIFO by default; just-in-time insertion only for at-risk, saveable URLLC packets
Link-State Awareness	None; the analytical rule assumes the average service rate	Infers slow-link periods from recent service times
Deadline-Risk Estimate	None, or a point estimate from the analytical rule	Calibrated upper bound with target coverage $1 - \alpha$
When It Intervenes	Never (FIFO) or for every URLLC packet (strict priority)	Only when a miss is predicted and the packet can still be saved
How Far a Packet Moves	Not at all, or to the head of the queue	To the latest position that still meets its deadline

URLLC p95/p99/p99.9 Delay and Miss Rate	Baseline values to be measured	Target: close to strict priority; to be measured
Bulk Traffic Cost	None under FIFO; highest under strict priority	Target: lower than strict priority and EDF; reported as cost per saved packet
Tuning	None	One main parameter, α, tied to the reliability target

Solution Architecture & Design

The system is built from the following components:

- TrafficGenerator: produces URLLC and bulk packet arrivals from a seeded random source.
- ChannelModel: two-state Gilbert–Elliott link that sets the current service rate.
- Queue: finite buffer that supports FIFO append, insertion at a position, and drop when full.
- Server: transmits the packet at the head of the queue at the current service rate.
- FeatureExtractor: builds the feature vector from queue state and recent service history at each URLLC arrival.
- DelayBoundPredictor: quantile regression model with conformal calibration that returns an upper bound on delay.

- SavabilityClassifier: predicts whether an intervention will turn a miss into a deadline met.
- SchedulerPolicy: common interface implemented by FIFO, StrictPriority, EDF, AnalyticalRule and the learned policies.
- CounterfactualForker: copies simulator state and runs both branches to create savability labels (training only).
- MetricsCollector and ExperimentRunner: record per-packet outcomes and run all policies across seeds and load conditions.

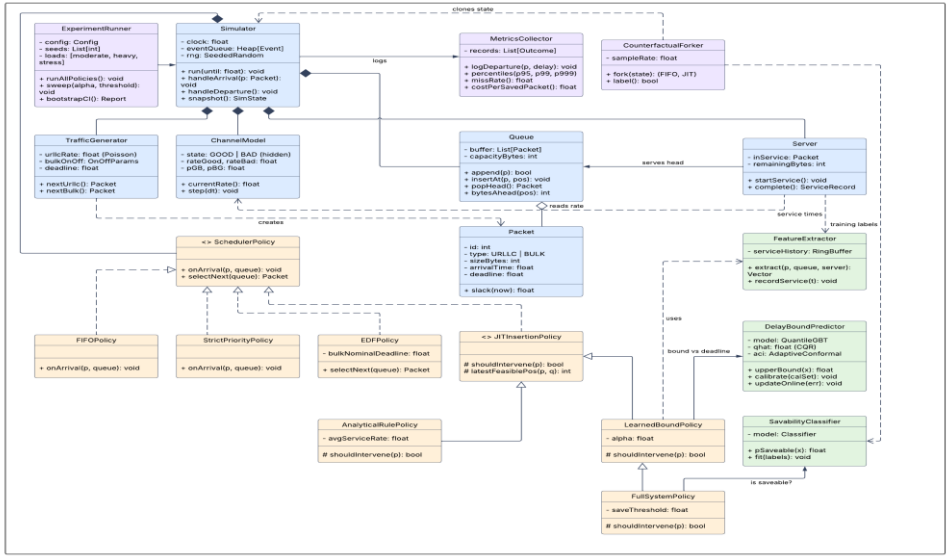


Figure: System Architecture UML Diagram

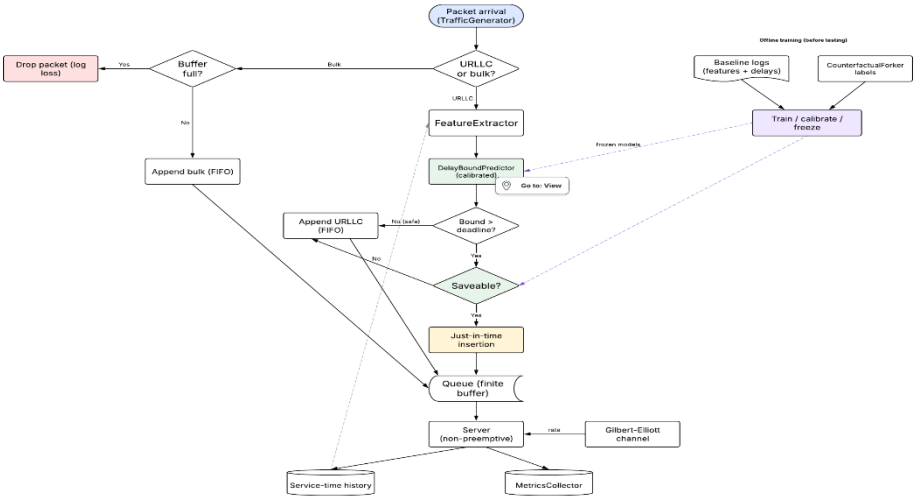
Data Flow Diagram

Packet-level flow at run time:

1. TrafficGenerator creates a packet arrival.
2. A bulk packet is appended to the queue in FIFO order, or dropped if the buffer is full.
3. For a URLLC packet, FeatureExtractor reads the queue state and recent service history.
4. DelayBoundPredictor returns a calibrated upper bound on the packet's delay.
5. If the bound is within the deadline, the packet is appended in FIFO order.
6. If the bound exceeds the deadline, SavabilityClassifier is queried. A saveable packet is inserted at the latest position that meets its deadline; otherwise it is appended in FIFO order.
7. Server transmits the head-of-queue packet at the rate set by ChannelModel. Each departure is logged by MetricsCollector, and its service time is added to the service history.

Offline training flow: baseline runs produce logged features and delays for the delay-bound model, and CounterfactualForker produces savability labels. Both models are trained on the training window, calibrated on the calibration window, and frozen before testing.

Figure: Data Flow Diagram (DFD)



Deployment Timeline

Deployment Milestones and Methodology

Our methodology is simulation-first: build and validate the simulator, generate data, train and calibrate the models, evaluate all policies, and document the findings. All milestones are planned work for the hackathon period. Core items are completed first; extensions are added once the core results are in place.

Milestone 1: Simulator and Baselines (Week 1)

- Objective: Implement and validate the discrete-event simulator with all non-learned policies.
- Tasks:
  - Build the finite-buffer, non-preemptive queue and the Gilbert–Elliott channel model.
  - Implement TrafficGenerator for URLLC and bulk traffic with explicit URLLC deadlines.
  - Implement FIFO, strict priority, EDF and the analytical rule with just-in-time insertion.
  - Validate against the M/G/1 mean delay with the channel held in one state, and check consistency (no negative delays, queue length matches arrivals minus departures and drops).
- Deliverable: simulator code, configuration file, validation report, and baseline delay and miss-rate plots for both loads.

Milestone 2: Data Generation and Model Training (Week 2)

- Objective: Generate datasets and train the calibrated delay-bound model.
- Tasks:
  - Run long baseline simulations to log features and delays at each URLLC arrival.
  - Split data chronologically into training, calibration and test windows.
  - Train the quantile regression model and apply conformal calibration.
  - Measure coverage, bound width and miss-detection quality against the analytical rule.
- Deliverable: trained and calibrated model files, and a prediction quality report (tests H1).

Milestone 3: Intervention and Evaluation (Week 3)

- Objective: Implement the learned policies and compare all six policies.
- Tasks:
  - Implement the learned-bound policy with just-in-time insertion.
  - Implement CounterfactualForker, generate savability labels and train the savability classifier.
  - Add adaptive conformal calibration and run the unseen stress condition.
  - Run all policies across independent seeds and both loads; sweep α and the savability threshold.
  - Produce trade-off curves, cost per saved packet, and confidence intervals for tail metrics.
- Deliverable: comparative graphs and a results summary (tests H2, H3 and H4).

Milestone 4: Documentation and Demonstration (Week 4)

- Objective: Finalize all deliverables for submission.
- Tasks:
  - Complete the technical report PDF following the template structure.
  - Set up the GitHub repository, add the AIORI account as a collaborator, and transfer ownership.
  - Build a demonstration dashboard where a reliability-target slider (α) shows URLLC misses and bulk cost changing for each policy.
  - Prepare a pseudocode walkthrough of the scheduler for the mentor.
- Deliverable: final report, repository, and live demonstration.

Risks and Fallbacks

- If counterfactual labelling is too slow, labels are generated on a sampled subset of URLLC arrivals.
- If the gradient-boosted model is unstable, linear quantile regression is used; calibration still provides the coverage guarantee.
- If time runs short, adaptive calibration and the stress condition are reported as extensions, and the core comparison of six policies is kept.

Bibliography I

[1] 3GPP, "Study on Scenarios and Requirements for Next Generation Access Technologies," 3GPP TR 38.913, 2017. [Online]. Available: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/
[2] E. N. Gilbert, "Capacity of a Burst-Noise Channel," *Bell System Technical Journal*, vol. 39, no. 5, pp. 1253–1265, 1960.
[3] E. O. Elliott, "Estimates of Error Rates for Codes on Burst-Noise Channels," *Bell System Technical Journal*, vol. 42, no. 5, pp. 1977–1997, 1963.
[4] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York: Springer, 2005.
[5] Y. Romano, E. Patterson, and E. Candès, "Conformalized Quantile Regression," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019.
[6] P. Gutierrez and J.-Y. Gérardy, "Causal Inference and Uplift Modelling: A Review of the Literature," in *Proc. International Conference on Predictive Applications and APIs*, PMLR vol. 67, pp. 1–13, 2017.
[7] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP Performance Metrics," RFC 2330, May 1998. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2330.html
[8] G. Almes, S. Kalidindi, M. Zekauskas, and A. Morton, Ed., "A One-Way Delay Metric for IP

Performance Metrics (IPPM)," STD 81, RFC 7679, January 2016. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7679.html
[9] J. Gettys and K. Nichols, "Bufferbloat: Dark Buffers in the Internet," *Communications of the ACM*, vol. 55, no. 1, pp. 57–65, 2012. [Online]. Available: https://doi.org/10.1145/2063176.2063196
[10] K. Nichols, V. Jacobson, A. McGregor, Ed., and J. Iyengar, Ed., "Controlled Delay Active Queue Management," RFC 8289, January 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8289.html
[11] B. Briscoe, Ed., K. De Schepper, M. Bagnulo, and G. White, "Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture," RFC 9330, January 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9330.html
[12] N. Finn, P. Thubert, B. Varga, and J. Farkas, "Deterministic Networking Architecture," RFC 8655, October 2019. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8655.html
[13] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973. [Online]. Available: https://doi.org/10.1145/321738.321743
[14] A. Demers, S. Keshav, and S. Shenker, "Analysis and Simulation of a Fair Queueing Algorithm," *ACM SIGCOMM Computer Communication Review*, vol. 19, no. 4, pp. 1–12, 1989. [Online]. Available: https://doi.org/10.1145/75246.75248

[15] N. C. Luong et al., "Applications of Deep Reinforcement Learning in Communications and Networking: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3133–3174, 2019. [Online]. Available: https://doi.org/10.1109/COMST.2019.2916583
[16] I. Gibbs and E. Candès, "Adaptive Conformal Inference Under Distribution Shift," in *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.

Thank You for Your Attention!